BrightLearn
DATA ANALYTICS

EXERCISE 02 · SQL FOUNDATIONS

SQL Aggregate Functions & Operators

A hands-on, pen-and-paper practice exercise. Read each table, write the SQL query that returns the requested data, and draw the expected output table.

COURSE
BrightLearn Basic SQL

FORMAT
Handwritten · Pen & Paper

QUESTIONS
15 queries across 5 tables

FOCUS
Aggregations · Filtering · Grouping

Read · Query · Draw
Learn SQL by writing it.

00 Exercise Instructions

This exercise builds on what you have learned about SELECT and filtering, and adds the most useful aggregate functions and operators in SQL. Work through each question carefully — accuracy beats speed.

★ LEARNING OBJECTIVES

By the end of this exercise you will be able to:

- Use aggregate functions: **COUNT, SUM, AVG, MIN, MAX**.
- Group results with **GROUP BY** and filter groups with **HAVING**.
- Apply common SQL operators: **DISTINCT, BETWEEN, IN, NOT, AND, OR**.
- Sort and limit results with **ORDER BY** and **LIMIT**.
- Predict the exact output of a query before running it.

FOR EACH QUESTION, DO THIS:

1

WRITE THE QUERY

Hand-write the full SQL statement that produces the requested result. Use proper SQL keywords in CAPS.

2

DRAW THE OUTPUT

Draw the expected output table by hand — column headers on top, rows underneath.

3

CHECK COLUMNS

Make sure your output uses the exact column names listed under *Expected Columns*.

SUBMISSION RULES

- **Format:** Handwritten on lined paper, using pen (no pencil, no typing).
- **Layout:** Number every answer to match the question (1 through 15).
- **Queries:** Write each SQL statement neatly. SQL keywords in `UPPERCASE`, table and column names in lowercase.
- **Output tables:** Draw clear column headers, then list each row beneath. Use a ruler if your handwriting needs help staying straight.
- **Originality:** All work must be your own — this exercise is graded on your understanding, not a perfect answer.

01 The students Table

A list of students enrolled at the academy, with their age and department.

▸ TABLE students

student_id	name	age	department
1	Alice	20	IT
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

01 List all distinct departments in the students table.

EXPECTED COLUMNS: `department`

02 Get the average age of students per department.

EXPECTED COLUMNS: `department` `avg_age`

03 Show departments with more than 1 student.

EXPECTED COLUMNS: `department` `student_count`

04 Get all students whose age is between 21 and 23.

EXPECTED COLUMNS: `student_id` `name` `age` `department`

05 List all students in the IT or HR department who are older than 21.

EXPECTED COLUMNS: `student_id` `name` `age` `department`

02 The courses Table

A catalogue of the courses offered by each department, along with the credits awarded.

▸ TABLE courses

course_id	course_name	department	credits
101	SQL Basics	IT	3
102	Python	IT	4
103	Data Science	IT	4
104	Excel	Finance	2
105	Statistics	HR	3

06 Show total credits per department, only for departments with more than 5 total credits.

EXPECTED COLUMNS: `department` `total_credits`

07 List all courses that do not have 4 credits.

EXPECTED COLUMNS: `course_id` `course_name` `department` `credits`

08 Show the top 3 courses by credits in descending order.

EXPECTED COLUMNS: `course_id` `course_name` `credits`

★ REMINDER — GROUP BY VS WHERE VS HAVING

WHERE filters individual rows *before* they are grouped. **GROUP BY** collects rows that share the same value into a group. **HAVING** filters whole groups *after* the grouping has happened. Question 6 needs all three working together — read it carefully before writing your query.

03 The enrollments Table

A record of which students are enrolled in which courses, with the grade each student received.

▸ TABLE enrollments

enrollment_id	student_id	course_id	grade
1	1	101	85
2	2	102	78
3	3	103	90
4	4	104	88
5	5	105	82

09 Get the maximum, minimum, and average grade across all enrollments.

EXPECTED COLUMNS: `max_grade` `min_grade` `avg_grade`

10 Count how many enrollments exist per course.

EXPECTED COLUMNS: `course_id` `enrollment_count`

★ ALIASING YOUR RESULTS

Notice that the expected column names (`max_grade`, `min_grade`, `avg_grade`, `enrollment_count`) do not appear directly in the table — they are **aliases**. Use the **AS** keyword in your queries so your output uses these exact names. For example: `MAX(grade) AS max_grade`.

04 The salaries Table

A simple payroll record showing salary and bonus amounts per employee, organised by department.

▸ TABLE salaries

employee_id	name	department	salary	bonus
1	Tom	IT	60000	5000
2	Jerry	HR	55000	4000
3	Spike	Finance	70000	6000
4	Tyke	IT	62000	5500
5	Butch	HR	54000	3500

11 **Find total salary and total bonus per department.**

EXPECTED COLUMNS: `department` `total_salary` `total_bonus`

12 **Show departments where average salary is above 55,000.**

EXPECTED COLUMNS: `department` `avg_salary`

13 **List employees whose salary plus bonus is greater than 60,000.**

EXPECTED COLUMNS: `employee_id` `name` `salary` `bonus` `total_compensation`

05 The projects Table

The list of active projects in the company, the department running each one, and the budget allocated.

▸ TABLE projects

project_id	project_name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
3	Dashboard	IT	150000
4	Website	Marketing	60000
5	HR Portal	HR	50000

14 **Show total and average budget per department. Only include departments with average budget above 70,000.**

EXPECTED COLUMNS: `department` `total_budget` `avg_budget`

15 **List all projects with budgets between 50,000 and 120,000, excluding the Marketing department.**

EXPECTED COLUMNS: `project_id` `project_name` `department` `budget`

★ BEFORE YOU SUBMIT — CHECK YOUR WORK

- Did you write all **15** SQL queries?
- Does each output table use the exact column names listed under *Expected columns*?
- Did you draw a clear output table for every query — even the ones with a single row?
- Did you use `AS` to alias columns where the question asks for a new name?
- Are your queries readable — keywords in CAPS, one clause per line where helpful?

Now write the SELECT statements.

The fastest way to learn SQL is to write it by hand and predict the output before running it. Take your time, sketch your tables clearly, and double-check your column names. You've got this.

★ BRIGHTLEARN DATA ANALYTICS ★